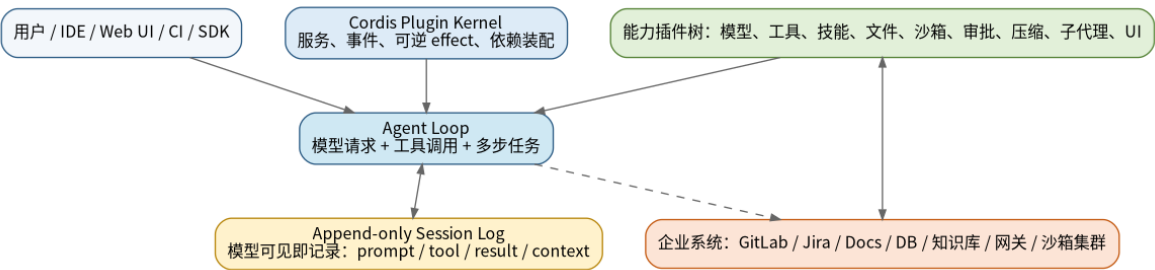


DeepSeek Harness 发布深度调查与企业内部 AI 集成借鉴报告

插件化 Agent Runtime、事件溯源、权限沙箱与 Skill Hub 的平台化启示

研究时间：2026-08-17 | 基于官方文档、GitHub、开发者讨论与公开报道



核心判断

DeepSeek Harness 的价值不在于“又一个代码 Agent UI”，而在于把 Agent 的模型、工具、技能、会话、沙箱、存储、循环、调度、UI 都抽象为可插拔能力，并用可追溯的 session log 把每一次模型可见输入、工具调用与上下文注入固化下来。这非常适合企业把 AI 集成从“脚本拼接”升级为“可治理的 Agent Runtime”。

0. 信息来源与可信度说明

本报告优先采用官方发布页、GitHub README、架构文档、开发者文档、Cordis 论文，以及 Anthropic/OpenAI/MCP 官方资料。公开媒体与 HN/Reddit 只用于观察社区反应，不作为技术事实的唯一依据。DeepSeek Harness 目前仍是 developer preview，官方和作者都提示接口会快速变化，因此本报告把“架构思想”与“当前实现细节”分开处理。

编号	来源	用途	可信度
S1	DeepSeek Harness 官方发布页 deepseek.com/harness/en/	发布定位、核心口号、运行模式、可追溯性	高
S2	deepseek-ai/deepseek-harness GitHub README	开源状态、运行方式、MIT、开发者预览	高
S3	docs/architecture.md	Cordis、profiles/bundles、核心包、事件、turn flow、session log、capability seams	高
S4	docs/tool-execution-pipeline.md	工具执行流水线、权限、审批、sandbox、pre/execute/post hooks	高
S5	DeepSeek Harness Providers guide	模型配置、自定义 OpenAI-compatible provider、密钥处理	高
S6	DeepSeek Harness Python SDK guide	无 UI/headless 使用、JSONL 会话日志、最小工具组合、风险提示	高
S7	DeepSeek Harness plugin tutorial	插件 apply、inject、ctx.effect、依赖和卸载清理	高
S8	Cordis paper: A Programming Paradigm for Spatiotemporal Composability	可逆 effect、reactive coeffect、动态组合理论背景	中高：预印本
S9	The Register / SCMP / The New Stack	发布背景、产业意义与第三方观察	中
S10	Hacker News thread	作者口径：early preview、MIT、rough edges、breaking changes；社区问题	中
S11	Anthropic Claude Code docs	hooks、subagents、skills、permissions 的对照参考	高
S12	OpenAI Codex AGENTS.md / MCP specification	项目指令、工具协议标准化的对照参考	高

1. 一页结论：如何看待 DeepSeek Harness 发布

DeepSeek Harness 应被看作“Agent 基础设施层”的一次公开样板，而不是单纯的聊天界面或代码生成器。官方表述是 Agent = Model + Harness：模型负责推理，harness 负责理解环境、使用工具、维持会话、执行任务与恢复现场。

- 第一，AI 竞争正在从“单模型能力”扩展到“模型 + 运行时 + 工具生态 + 组织流程”。企业真正的壁垒会逐渐从 prompt 积累，转向可插拔工具、可审计轨迹、可控权限、可复用技能和评测闭环。
- 第二，DeepSeek Harness 的架构关键词是“everything is a plugin”。模型、工具、技能、session log、agent loop、UI、sandbox、storage、scheduler 都可以替换或叠加。这使企业可以按业务线组合不同 Agent，而不是为每个项目复制一套代码。
- 第三，append-only session log 是关键。它把模型看到的一切记录下来，支撑 resume、fork、search、replay、审计、成本分析、事故复盘和离线评测。企业内集成 AI，最怕“结果对了但过程不可解释”；Harness 的事件溯源思路值得优先借鉴。
- 第四，工具执行不是简单 function calling，而是一条可治理流水线：pre-execute、approval、guard、execute、post-execute、finalize、tool/result。权限、审批、沙箱、超时、重试、指标、结果改写都在模型之外强制执行。
- 第五，目前它仍是 developer preview。适合做源码学习、POC、架构启发、内部平台原型；不建议在没有封装层、评测和安全治理的情况下直接承载生产核心流程。

给公司内部 AI 集成的直接建议

不要把 Harness 当成“买来即用的工具”。更合理的路径是：抽取其架构思想，建设内部 Agent Runtime：模型网关 + 插

2. 发布背景：为什么 Harness 重要

过去一年，Claude Code、Codex CLI、OpenCode、Cursor/Agent IDE、MCP 生态不断推动“代码 Agent”从聊天工具变成工程操作平台。Harness 的出现说明 DeepSeek 也在向 agentic AI 纵深推进：模型本身之外，还需要一个能组织工具、上下文、权限、文件、子任务和 UI 的运行时。

从官方发布页看，Harness 直接把能力范围扩展到 models、tools、skills、sessions、sandboxes、storage、loops、scheduling、UI，并把它们全部作为插件处理 [S1]。这说明其目标不是只服务某一个模型，而是要成为一层可组合的 agent runtime。

GitHub README 明确其为开源项目、MIT license，并提示当前是 developer preview，未来会有兼容性破坏变更 [S2]。HN 上一位自称作者的评论也强调这是早期 developer preview，会有 rough edges 和 compatibility-breaking changes [S10]。因此，本报告的重点不是“今天能否替代 Claude Code”，而是“这套架构是否值得公司内部吸收”。

3. DeepSeek Harness 是什么：一句话与三层拆解

一句话：DeepSeek Harness 是一个面向 Agent 的可插拔运行时，把模型、工具、会话、技能、权限、沙箱、UI 和调度组合成可运行、可追溯、可扩展的 Agent。

层级	核心问题	Harness 的做法	企业可借鉴点
模型层	用哪个模型、如何切换、如何适配自建网关	LLM adapter seam；官方模型、自定义 provider、OpenAI-compatible endpoint；模型配置下次请求生效 [S5]	内部统一模型网关，不让业务代码直接绑定某一家模型 API
能力层	Agent 能做什么、工具如何治理	tools/skills/sandbox/approval/filesystem/subagent 都是插件；工具执行走统一 pipeline [S3][S4]	把 GitLab、Jira、DB、知识库、文件系统等封装成受控插件
轨迹层	过程如何追踪、失败如何恢复、结果如何审计	append-only session log；模型可见即记录；resume/fork/replay 从同一事件流派生 [S1][S3]	构建企业级 trace、审计、评测、事故复盘和数据集沉淀

4. 技术架构深拆

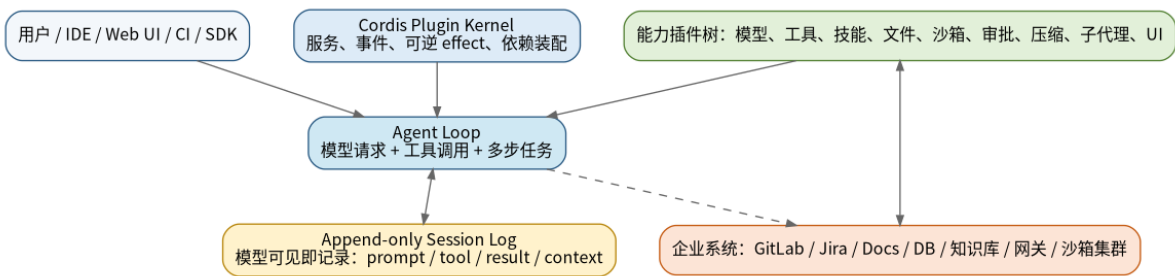


图 1：DeepSeek Harness 的核心不是某个固定 workflow，而是 Cordis 插件内核 + Agent loop + session/event log + capability seams。

4.1 Cordis：把动态组合变成一等公民

官方架构文档称 Cordis 是 dsh 底层框架：插件向共享 context 贡献 services、typed events、reversible effects；模型适配器、工具注册表、session log、agent loop 都是插件，因此可从配置中替换 [S3]。

这个设计解决了企业 AI 集成中最常见的两个问题：第一，工具越来越多之后，依赖关系混乱；第二，不同业务线需要不同能力组合，但又不想复制整套 Agent 代码。Cordis 的价值在于把“挂载、卸载、依赖、事件、清理”变成框架能力，而不是靠每个工程师手写 glue code。

Cordis / Harness 概念	普通插件系统	企业 AI 集成含义
可逆 effect	不只是注册，还能在卸载时回滚注册、监听器、定时器等副作用	插件热更新、安全回滚、灰度发布更可控
服务 + 依赖 inject	插件声明需要哪些服务，框架等待依赖就绪再加载	工具开发者不需要关心全局初始化顺序
typed events	事件成为扩展点，而非在核心代码里到处 patch	权限、审计、指标、策略可以旁路插入
配置组合	profile/bundle/patch 叠加，而不是改源码	不同团队可拥有不同 preset，但共享底层能力

4.2 Profiles、Bundles、Patches：把“环境差异”配置化

Harness 运行时是由有序 layers 组成的 plugin tree。profile 是命名组合；bundle 是 Cordis config rows 与代码分发格式；web 和 headless 是模板；dsh-base 提供模型适配器、工具、持久化、sandbox、approval policy、settings、credentials、telemetry；web-app 增加浏览器 UI；headless 增加一次性 runner [S3]。

这对企业内部非常重要：开发、测试、生产、CI、离线评测、数据敏感环境不应共用同一套硬编码配置。应当用 profile/preset 描述“此场景允许哪些模型、哪些工具、哪些权限、写入哪些目录、是否需要审批”。

4.3 Core packages：最小 Agent Runtime 的模块边界

核心包/服务	负责内容	企业内部可对标模块
core/session	append-only SessionEvent log 与内存 store [S3]	会话事件库、审计日志、重放数据集
core/system-prompt	prompt sections 和 tool schemas 组装 [S3]	系统提示模板、工具 schema、上下文注入器
core/tools	scoped tool registry 与受保护的执行 pipeline [S3]	企业工具注册中心与策略执行层
core/agent / agent-loop	Agent 接口、live registry、默认 driver [S3]	任务循环、状态机、子任务管理
llm/llm	消息、stream vocabulary、adapter seam [S3]	模型网关 SDK、provider 适配器、流式协议

4.4 Turn / Step 生命周期：为什么可控



官方文档把 step 定义为一次模型请求加上它调用的工具；turn 是零个或多个 step。关键事件包括 turn/start、agent/pre-step、step/start、assistant/chunk、tool/call、tools/pre-execute、tools/execute、tools/post-execute、tool/result、step/end、turn/end [S3][S4]。

这套生命周期的本质是：模型“想做什么”与系统“允许它怎么做”被清晰分层。模型输出 tool call 只是意图，真正执行之前会经过权限、审批、沙箱、guard、timeout、retry、metrics、result normalization 等环节。企业中这比简单 prompt 约束更可靠。

4.5 Session Log：企业 Agent 最该复制的核心资产

官方强调 session log 是模型看到的上下文来源；deriveMessages() 从事件流投影模型历史；raw assistant/chunk 保留 replay 与 UI fidelity；fork、resume、transcripts、telemetry、persistence 都来自同一条流。更关键的原则是“Model-visible means logged”：凡是进入模型请求的内容必须能从日志重建，并由 runtime invariant 断言 [S3]。

企业落地启示

不要只保存最终回答。必须保存：系统提示、用户输入、检索片段、工具 schema、工具参数、工具结果、审批结果、模型输出、错误、重试、上下文压缩、子代理轨迹。否则后续无法评测、审计、复盘、合规，也无法构建公司自己的 Agent 数据集。

4.6 Capability seams：从“接 API”变成“可替换能力面”

Harness 把 seam 定义为可替换能力，包含 Service Definition、Service Provider、Consumer 三个角色。文档举例：文件系统与 subprocess providers 共用一个执行世界，因此换成远程 sandbox 时 Bash、PTY、LSP 可一起迁移；subagent providers 也可以在同一接口后面实现为子 Agent 或委托给其他产品 [S3]。

这个模式适合公司内部集成：不要把 GitLab 工具、数据库工具、浏览器工具、文件系统工具写成一个大类。先定义能力接口，再允许本地、远程、沙箱、只读、审计版等多个 provider 共存。

4.7 工具执行 Pipeline：把安全、审批、审计放在模型之外

工具 pipeline 显示：assistant message 中的 tool-call 先写入 tool/call session event，再进入 UI pending card、tools/pre-execute、monotonic guards、approval、tools/execute、工具体、fs/write-intent 或 fs/edit-intent、tools/post-execute、finalizeContent、tools/result、tool/result session event [S4]。

环节	作用	企业控制点
tools/pre-execute	hooks、permission、sandbox，可直接 allow/deny/ask	敏感工具白名单、参数检查、审批发起
guards	单调拒绝或放行，保护身份与不可重排策略	强制安全边界：禁止写生产库、禁止外发密钥
approval	一次性人工确认，无法确认时 deny	高风险命令、删除、部署、发邮件需人工确认
tools/execute	timeout、retry、metrics 包裹实际工具体	可靠性与可观测性
fs write/edit intent	文件修改意图进入文件系统策略	只允许工作区写入、禁止修改配置密钥
post-execute	接受、阻断、替换、追加上下文	结果脱敏、输出截断、错误归因、补充解释
tools/result	冻结权威 outcome 后通知	审计日志、评测数据、UI 展示、成本统计

4.8 模型配置与自建网关

模型配置文档明确支持 DeepSeek、catalog provider、custom provider。自定义 provider 可配置 base URL、API protocol、credential 和模型列表；这对企业已有 OpenAI-compatible 网关、自托管 vLLM/SGLang、内部模型路由非常关键 [S5]。

密钥处理也有借鉴意义：页面保存后只返回 redacted descriptor，字面 secret 存在 \$DSH_HOME/.credentials.yaml，settings 只保留 credential reference [S5]。企业平台应进一步把这部分对接 Vault/KMS，并用 RBAC 限制插件读取密钥。

4.9 Python SDK 与 Headless：从工具到平台的入口

Python SDK 文档展示了一个程序化运行方式：使用 DeepSeekHarness 传入 provider、model、max_tokens、cwd、session_root、cordis 配置，然后 run(task, session_id)。示例说明 session 目录会收到 JSONL log，包含组装后的 model requests 与 tool calls；同一 session id 可保留 session-owned Bash 状态 [S6]。

这说明 Harness 不只适合交互式 UI，也能进入 CI、批处理、评测、离线数据生成。企业内部应优先建设 headless/SDK 调用方式，使 Agent 成为工程系统的一部分，而不是只能靠人打开网页操作。

4.10 Skills、Sandbox、Permission、Compaction

能力	Harness 当前思路	企业内部设计建议
Skills	skill capability family 包含 registry、filesystem provider、badge provider、model-facing skill tool；skills 是可选 instructions，不是 session event [S7]	建设 Skill Hub：每个 skill 有版本、权限、依赖、测试、签名、评测结果、适用团队
Sandbox	sandbox seam 包裹 same-world subprocess argv；模式控制文件系统效果：read-only、workspace-write、danger-full-access；网络和进程可见性不在该词汇内 [S7]	不能只依赖本地 sandbox；敏感任务用容器/远程沙箱；网络 egress 必须独立管控
Permission presets	把 sandbox mode 与 approval policy 组合成 named presets；默认包括 workspace-write+ask 与 danger-full-access+never [S7]	按场景定义 preset：阅读、改代码、CI 修复、部署、数据查询、生产操作
Compaction	作为可选 capability seam，不在 agent-loop 主干；对 session 和 LLM 依赖较强 [S7]	把上下文压缩作为可替换策略：摘要、裁剪、检索重构、token/cost aware

5. 与 Claude Code / Codex / MCP 的对比

DeepSeek Harness 并不是在空白处出现。Claude Code 已经把 hooks、subagents、skills、permissions、SDK 做得很成熟；OpenAI Codex 强调 AGENTS.md、本地/云端编码 Agent 与 MCP/Agents SDK workflow；MCP 则提供工具/上下文连接协议。Harness 的不同点在于，它更明确地把“整个 Agent 运行时本身”作为插件组合系统来设计。

维度	DeepSeek Harness	Claude Code	OpenAI Codex / AGENTS.md	MCP
定位	开源 agent harness / runtime，everything is plugin [S1][S2]	成熟商业 coding agent 与 SDK，hooks、subagents、skills、permissions 丰富 [S11]	本地/云 coding agent；AGENTS.md 提供项目指令层 [S12]	标准化工具/数据源接入协议 [S12]
扩展方式	Cordis plugins、services、events、patch overlays [S3] [S7]	hooks、plugins/ marketplaces、skills、subagents、settings [S11]	AGENTS.md + MCP + CLI/SDK 集成 [S12]	server exposes tools/resources/prompts
可追溯性	append-only session log，模型可见即记录 [S3]	headless stream 可包含 subagent parent_tool_use_id 等，适合重构部分轨迹 [S11]	Codex/Agents SDK 可审计，但依实现而定	协议层不负责完整 Agent 轨迹
治理边界	工具 pipeline + guards + approval + sandbox [S4]	hooks/permissions/ workspace trust/managed settings 强 [S11]	依赖 CLI 策略与环境隔离	只提供交互协议，安全要靠 client/server 实现
企业启发	适合学习平台架构与内部 runtime	适合学习成熟 UX、权限、hooks、子代理	适合学习项目指令规范与研发 workflow	适合标准化工具协议

判断：如果要马上提升研发效率，Claude Code/Codex 类成熟产品更直接；如果要建设公司自己的 Agent 底座，Harness 的可组合 runtime、session log 和 capability seam 更值得系统研究。

6. 对公司内部 AI 集成的核心借鉴思想

总原则

公司内部不要追求“一个万能 Agent”。应建设“一个稳定 Agent Runtime + 多个业务 profile + 多个受控插件 + 可复用 Skill 市场 + 完整轨迹审计”。

6.1 插件优先，而不是功能堆叠

把模型、工具、知识库、数据库、文件系统、浏览器、CI、权限、审批、沙箱、压缩、子代理都做成插件能力。新增能力只注册到明确 extension point，不改核心 loop。

6.2 Session/Event Log 优先

每次模型请求、工具调用、工具结果、审批、上下文注入、错误、重试都写成事件。最终回答只是事件流的投影。这样才能支撑 replay、fork、审计、评测、事故复盘和数据集沉淀。

6.3 能力接口与 provider 分离

先定义服务接口，再实现本地 provider、远程 provider、沙箱 provider、只读 provider、生产受控 provider。模型-facing tool 只依赖接口，不直接依赖具体系统。

6.4 安全边界写在 pipeline，不写在 prompt

prompt 只能影响模型倾向，不能构成强制边界。必须在 tools/pre-execute、guard、approval、sandbox、post-execute 层做强制策略。

6.5 Profile/Preset 管理环境差异

不同场景用不同 profile：阅读型 Agent、代码修复 Agent、CI Agent、数据分析 Agent、生产运维 Agent。每个 profile 明确模型、工具、写权限、审批、网络、日志留存。

6.6 Skill Hub 要从“提示词仓库”升级为“流程资产仓库”

每个 skill 不只是 SKILL.md，还应包括版本、依赖、权限声明、测试用例、评测结果、适用模型、输入输出契约、风险等级、回滚策略。

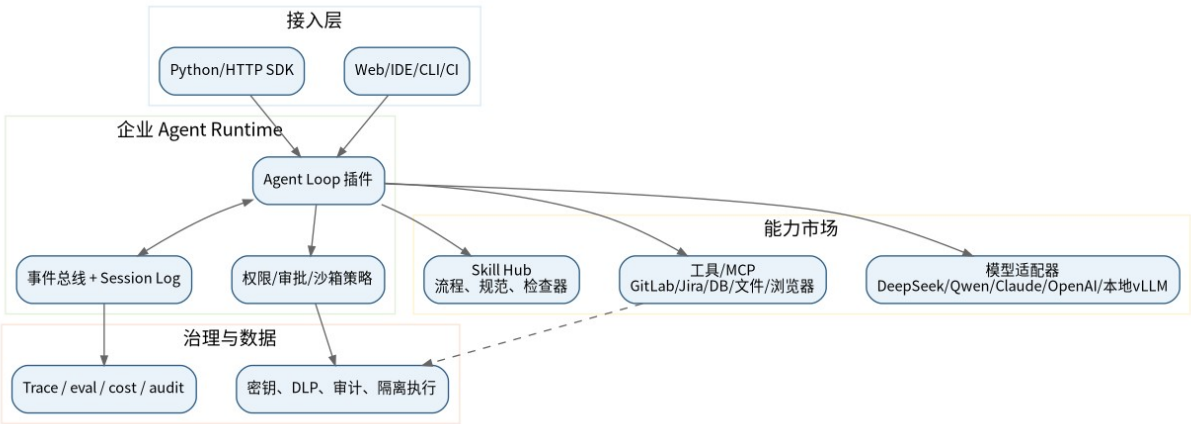
6.7 让评测和观测成为默认功能

每个 session 产生 trace；每个插件产生指标；每个 skill 有回归测试；每次模型/工具升级都能 replay 历史任务。

6.8 构建 Headless/SDK，而不是只做 Chat UI

CI、批处理、自动修复、知识库刷新、数据报表、PR review 都需要程序化入口。Web UI 是人机交互层，不应成为唯一入口。

7. 企业内部参考架构方案



建议把内部 AI 平台拆成六层：接入层、Agent Runtime、能力市场、模型网关、治理安全、评测观测。

层	模块	职责	关键设计
接入层	Web / IDE / CLI / CI / Python SDK	让人和系统都能调用 Agent	统一任务 API、session id、workspace、profile、权限 preset
Agent Runtime	Loop + Event Bus + Session Log	驱动模型与工具多步交互	事件溯源、可 replay、可 fork、可 resume
能力市场	Tools / MCP / Skills / Subagents	复用内部系统与流程知识	插件签名、版本、依赖、权限声明、自动测试
模型网关	OpenAI-compatible gateway	统一模型、计费、限流、fallback	DeepSeek/Qwen/Claude/OpenAI/本地 vLLM/SGLang 可路由
治理安全	Sandbox / Approval / RBAC / DLP	限制 Agent 真实世界副作用	workspace-write 默认、生产操作审批、网络 egress 控制
评测观测	Trace / Eval / Cost / Audit	持续改进和合规	每次执行留 trace；历史任务回放；按 skill/model/profile 统计

8. 典型落地场景设计

场景	Profile 配置	关键插件	成功指标	风险控制
代码审查 / PR Review	read-only 或 workspace-write+ask	GitLab、diff reader、test runner、code search、skill/code-review	发现缺陷率、误报率、review 时间	禁止直接 push；敏感文件只读；所有建议落 trace
CI 自动修复	workspace-write+ask，独立容器	bash、editor、test、package manager、GitLab MR	修复通过率、平均迭代轮数、成本	只允许分支写入；依赖安装白名单；超时和资源限制
内部知识问答	read-only	RAG、文档库、权限过滤、引用生成	答案命中率、引用覆盖率、幻觉率	按用户权限过滤；答案必须可引用；不写入业务系统
数据分析 / 报表	read-only DB + 文件写入	SQL 工具、BI、Python、表格导出	报告交付时间、SQL 正确率	SQL 只读；行数限制；结果脱敏；高风险查询审批
运维辅助	默认 read-only，生产变更 ask/deny	日志、监控、K8s、runbook skill	MTTR、定位准确率	生产写操作必须人审；命令白名单；网络限制
Skill 编译/验证	headless minimal profile	skill linter、schema checker、sandbox test、eval replay	skill 通过率、回归失败率	skill 作为供应链资产扫描；禁止隐式外联和密钥访问

9. 针对 Skill Hub / GitLab Skill 的具体设计建议

如果公司已有或计划建设 GitLab-based Skill Hub，应将 Harness 的插件化与 session log 思路结合：Skill 不应只是 Markdown 指令，而应是可安装、可测试、可审计、可回滚的能力包。

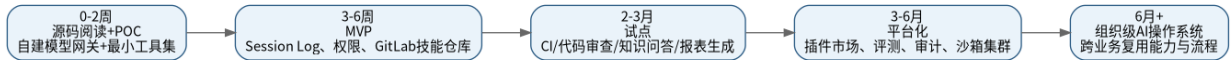
设计项	建议
Skill 包结构	SKILL.md + manifest.yaml + permissions.yaml + tests/ + examples/ + eval_cases/ + changelog.md + signing metadata
权限声明	声明需要哪些工具、文件路径、网络域名、密钥、模型能力；安装时生成风险等级
编译/验证	不是把 prompt 编译成二进制，而是把 skill 转成“规范化能力描述 + schema + 测试快照 + 静态检查报告”
兼容矩阵	记录适用模型、适用 profile、依赖插件版本、最低工具版本、已验证平台
安全扫描	扫描 shell 命令、外部 URL、密钥路径、obfuscated command、危险文件路径、数据外发
评测回放	每次 skill 更新 replay 历史 session/task；对比成功率、步骤数、成本、错误类型
发布流程	MR 审核 + 自动测试 + 签名 + 内部 marketplace 可见性 + 灰度启用 + 回滚
运行时注入	不要永久塞进 system prompt；按任务 relevance 注入，注入内容也

	写 session event
--	-----------------

10. 风险、限制与反直觉点

风险	表现	缓解策略
Developer preview 不稳定	API、插件、配置可能 breaking change	先做隔离封装层；不要把业务逻辑直接耦合 upstream API；锁版本
插件供应链风险	Skill/插件可引导工具调用、访问文件或外联	签名、静态扫描、权限声明、受控 marketplace、最小权限
Sandbox 误解	文件系统沙箱不等于网络/进程/容器隔离	高风险任务使用容器/远程隔离；网络 egress 单独控制
日志隐私风险	完整 session log 可能含代码、密钥、个人信息、客户数据	DLP、脱敏、访问控制、保留期、加密、审计
工具结果污染上下文	工具返回过长/恶意/不可信内容会影响模型	结果截断、内容标注、来源评级、prompt injection 检测
模型可替换但行为不可等价	同一 harness 换模型后表现、工具调用风格会变	按模型/profile 做评测；关键流程固定模型或灰度
过度平台化	一开始就做大而全，拖慢业务落地	先选 2-3 个高价值场景，平台能力随场景生长

11. 实施路线图



阶段	目标	交付物	验收标准
0-2 周 POC	验证 Harness 架构与内部网关/工具适配可行性	源码阅读笔记、最小 profile、自建 OpenAI-compatible provider、2 个工具插件	能在隔离 repo 中完成代码阅读/测试/修改；产生可 replay JSONL trace
3-6 周 MVP	建设公司 Agent Runtime 原型	session log schema、权限 preset、GitLab/Docs/DB 只读插件、Skill Hub v0	至少 3 个真实任务跑通；权限审批可控；trace 可查询
2-3 月 试点	接入研发与知识问答场景	PR review、CI 修复、知识库问答、报表生成 profile	成本、成功率、人工节省、误报率有仪表盘
3-6 月 平台化	插件市场、评测回放、沙箱集群	插件签名、skill 验证、远程容器执行、模型评测矩阵	业务线可自助发布插件/skill；升级前自动 replay
6 月+ 组织级体系	把 AI 能力沉淀为组织资产	Agent Runtime 标准、内部 marketplace、治理制度、最佳实践	跨团队复用率提升；核心流程可审计可回滚

12. 推荐的内部技术选型组合

模块	推荐做法	说明
模型网关	OpenAI-compatible gateway + provider registry	屏蔽 DeepSeek/Qwen/Claude/OpenAI/本地模型差异；统一鉴权、限流、成本
事件存储	PostgreSQL / ClickHouse + 对象存储	Postgres 存 session/event 元数据；对象存储保存大 tool result/文件快照；ClickHouse 做分析
工具协议	MCP + 内部 tool SDK 并行	对外优先 MCP；内部高权限工具走专用 SDK 和强策略
执行隔离	本地 read-only + 容器 workspace-write + 远程 sandbox	开发低风险本地；CI/生产远程隔离；网络 egress 白名单
Skill Hub	GitLab monorepo/group + marketplace index	利用 GitLab 权限/MR/CI；产出索引供 Agent Runtime 安装
评测	session replay + golden tasks + fault injection	每次模型、工具、skill、prompt 更新都能回放真实任务

可观测	OpenTelemetry-like trace + 自定义事件	记录成本、延迟、工具调用、失败类型、审批、token、模型版本
安全	Vault/KMS + DLP + RBAC + audit	密钥不进 prompt；日志脱敏；敏感工具最小权限

13. 最小可行内部 Harness 设计草案

下面是一套不依赖 DeepSeek Harness 代码、但借鉴其思想的内部 MVP。适合公司已有模型网关、GitLab、Skill Hub、代码仓库和内部知识库的情况。

组件	MVP 版本	后续增强
AgentRuntime	Python/TypeScript 服务，维护 turn/step 状态机	Cordis-like 插件系统、热插拔、分布式调度
SessionEvent	JSONL + Postgres 索引，记录 user/message、assistant/message、tool/call、tool/result	完整 event schema、压缩、fork/replay、隐私分级
ToolRegistry	注册工具 schema、执行函数、权限声明	多 provider、MCP adapter、工具版本化
PolicyPipeline	pre/execute/post 三段钩子；默认 read-only	approval、guard、DLP、result rewrite、异步 job
SkillRegistry	从 GitLab 拉取 SKILL.md 与 manifest	测试、签名、依赖、自动评测、灰度
ModelAdapter	统一 OpenAI-compatible chat/stream	多模型路由、fallback、cost-aware、上下文策略
Sandbox	Docker workspace，挂载 repo 副本	K8s/Firecracker、网络控制、资源配额
UI/SDK	CLI + Python SDK + 简单 Web Trace viewer	IDE 插件、CI Bot、企业权限中心

14. 决策建议

- 短期不要押注 Harness 作为生产依赖；应先把它作为“架构样板 + 源码学习对象 + POC 组件”。
- 优先复制 session/event log 与 tool pipeline。它们是企业级 Agent 能否治理的分水岭。
- 把现有 Skill Hub 与 Harness 的 plugin/profile/preset 思想合并：Skill 要有权限声明、测试、评测、签名和回放。
- 围绕 2-3 个真实高价值场景试点：代码审查、CI 自动修复、内部知识问答/报表。不要一开始泛化到所有业务。
- 用内部模型网关统一 DeepSeek、Qwen、Claude/OpenAI、local vLLM/SGLang。Agent Runtime 不应依赖单一模型厂商。
- 建立“模型/工具/Skill 升级前 replay”的工程纪律。没有 replay，就无法知道一次升级是否让 Agent 更好还是更危险。
- 安全策略从 prompt 外移到 runtime：权限、审批、沙箱、网络、DLP、密钥、日志访问都必须由系统强制。

15. 参考链接

- S1. DeepSeek Harness official landing: <https://deepseek.com/harness/en/>
- S2. GitHub README: deepseek-ai/deepseek-harness: <https://github.com/deepseek-ai/deepseek-harness>
- S3. DeepSeek Harness Architecture: <https://raw.githubusercontent.com/deepseek-ai/deepseek-harness/refs/heads/master/docs/architecture.md>
- S4. Tool Execution Pipeline: <https://raw.githubusercontent.com/deepseek-ai/deepseek-harness/refs/heads/master/docs/tool-execution-pipeline.md>
- S5. Configure models guide: <https://deepseek-harness.github.io/deepseek-harness/en/guide/providers>
- S6. Python SDK guide: <https://deepseek-harness.github.io/deepseek-harness/en/guide/python-sdk>
- S7. Plugin tutorial and subsystem docs: <https://deepseek-harness.github.io/deepseek-harness/en/develop/basic/>
- S8. Cordis paper: <https://github.com/cordiverse/paper>

- S9. The Register / SCMP / The New Stack reports: <https://www.theregister.com/ai-and-ml/2026/08/14/deepseeks-innovative-harness-treats-everything-as-a-plugin/>
- S10. Hacker News discussion: <https://news.ycombinator.com/item?id=49285244>
- S11. Anthropic Claude Code docs: <https://docs.anthropic.com/en/docs/claude-code/hooks-guide>
- S12. MCP specification and OpenAI Codex AGENTS.md: <https://modelcontextprotocol.io/specification/2026-07-28>